HW 4

Name: Chaitany Raghav
TSN: 200478852

Q1.3

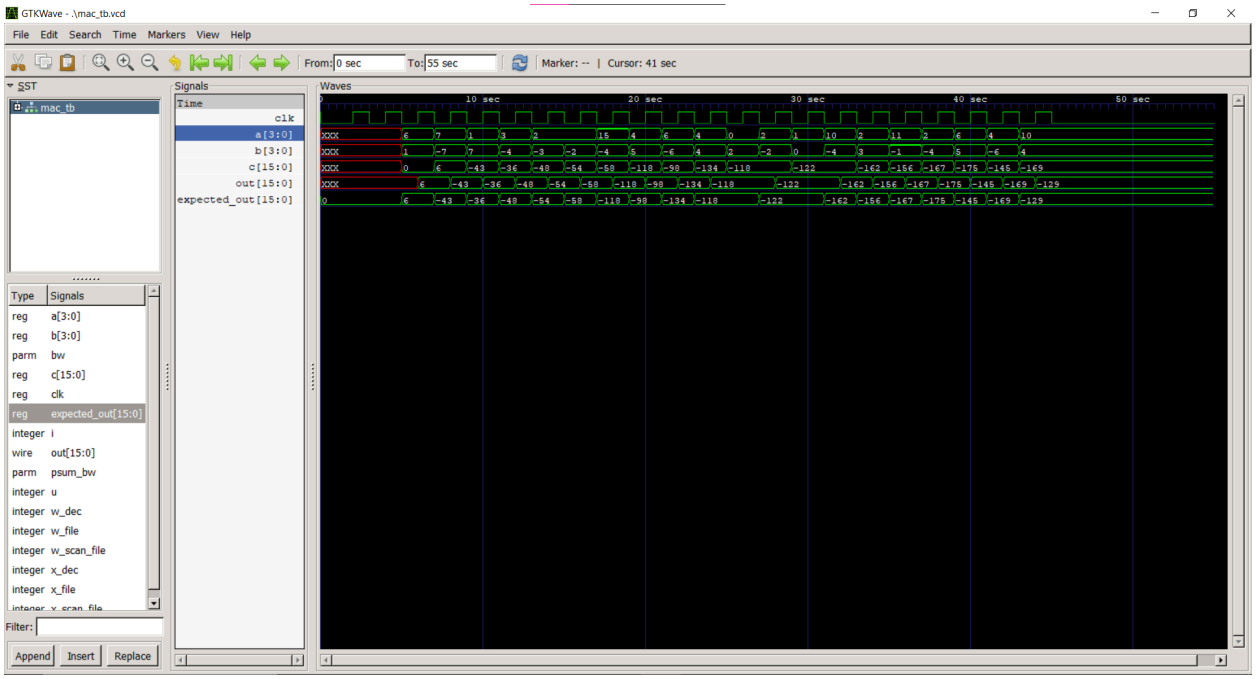
Observations:

Model	Top - 1 Accuracy	Δ Accuracy
Baseline	92.79	-
8 Bit PQT	92.82	+0.03
8 Bit PQT with MSE-based α search	92.83	+0.04
4 Bit PQT	87.73	-5.06
4 Bit PQT with MSE-based α search	89.19	-3.60

- **8-Bit Weight Quantization (+0.04%):** Compressing the weights to 8 bits gives the model 256 discrete levels to work with. This resolution is more than enough to capture all the core features of the network. Interestingly, the slight rounding effect actually acts like a mild regularization filter. Smoothing out the tiny, noisy decimal variations in the original FP32 baseline helps reduce minor overfitting and allows the model to generalize slightly better on the test set.
- **4-Bit Weight Quantization (-3.60%):** Squeezing everything into 4 bits drops the resolution down to just 16 discrete levels. This harsh compression introduces significant quantization noise and creates an outlier issue. Without optimization, a few highly sensitive weight or activation spikes force the quantization scale to stretch too wide. This ends up crushing 99% of the normal weights into the same few bins, which completely strips away the model's ability to handle fine-grained details and causes a drop in accuracy.
- **The Alpha Strategy:** Implementing an alpha-parameter search essentially gives us a smart mathematical slider. It dynamically scales down the extreme outliers before the model gets quantized. For the 8-bit model, it fine-tuned an already stable setup to maximize the regularization benefits. For the 4-bit model, it was a lifesaver—protecting the most critical 1% of features from being wiped out entirely. This alone salvaged a massive chunk of performance, clawing back 1.46% of baseline accuracy to bring the drop down from -5.06% to a much more acceptable -3.60%.

Q 2

Waveform:



mac.v:

```
// Created by prof. Mingu Kang @VVIP Lab in UCSD ECE department
// Please do not spread this code without permission
module mac (out, a, b, c);

parameter bw = 4;
parameter psum_bw = 16;

input unsigned [bw-1:0] a;
input signed [bw-1:0] b;

input signed [psum_bw-1:0] c;

output signed [psum_bw-1:0] out;

wire signed [bw:0] a_signed;
wire signed [2*bw:0] product;
```

```
assign a_signed = $signed({1'b0, a});
assign product = a_signed * b;
assign out = c + product;

endmodule
```

mac_tb.v:

```
// Created by prof. Mingu Kang @VVIP Lab in UCSD ECE department
// Please do not spread this code without permission


module mac_tb;

parameter bw = 4;
parameter psum_bw = 16;

reg clk = 0;

reg  [bw-1:0] a;
reg  [bw-1:0] b;
reg  [psum_bw-1:0] c;
wire [psum_bw-1:0] out;
reg  [psum_bw-1:0] expected_out = 0;

integer w_file ; // file handler
integer w_scan_file ; // file handler

integer x_file ; // file handler
integer x_scan_file ; // file handler

integer x_dec;
integer w_dec;
integer i;
integer u;

function [3:0] w_bin ;
  input integer  weight ;
  begin
```

```
    if (weight>-1)
     w_bin[3] = 0;
    else begin
     w_bin[3] = 1;
     weight = weight + 8;
    end

    if (weight>3) begin
     w_bin[2] = 1;
     weight = weight - 4;
    end
    else
     w_bin[2] = 0;

    if (weight>1) begin
     w_bin[1] = 1;
     weight = weight - 2;
    end
    else
     w_bin[1] = 0;

    if (weight>0)
     w_bin[0] = 1;
    else
     w_bin[0] = 0;

  end
endfunction



function [3:0] x_bin ;
  input integer  weight ;
  begin
    for (u=bw-1; u>-1; u=u-1) begin
      if (weight>2**u-1) begin
        x_bin[u] = 1;
        weight = weight - 2**u;
      end
      else begin
```

```
        x_bin[u] = 0;
      end
    end
  end
endfunction


// Below function is for verification
function [psum_bw-1:0] mac_predicted;
    input [bw-1:0] a;
    input signed [bw-1:0] b;
    input signed [psum_bw-1:0] c;

    reg signed [bw:0] a_signed;
    reg signed [2*bw:0] product;

    begin
        a_signed = $signed({1'b0, a});
        product = a_signed * b;
        mac_predicted = c + product;
    end
endfunction



mac_wrapper #(.bw(bw), .psum_bw(psum_bw)) mac_wrapper_instance (
  .clk(clk),
        .a(a),
        .b(b),
        .c(c),
  .out(out)
);


initial begin

  w_file = $fopen("b_data.txt", "r");  //weight data
  x_file = $fopen("a_data.txt", "r");  //activation

  $dumpfile("mac_tb.vcd");
```

```
  $dumpvars(0,mac_tb);

  #1 clk = 1'b0;
  #1 clk = 1'b1;
  #1 clk = 1'b0;

  $display("-------------------- Computation start --------------------");


  for (i=0; i<20; i=i+1) begin  // Data lenght is 10 in the data files

     #1 clk = 1'b1;
     #1 clk = 1'b0;

     w_scan_file = $fscanf(w_file, "%d\n", w_dec);
     x_scan_file = $fscanf(x_file, "%d\n", x_dec);

     a = x_bin(x_dec); // unsigned number
     b = w_bin(w_dec); // signed number
     c = expected_out;

     expected_out = mac_predicted(a, b, c);

  end



  #1 clk = 1'b1;
  #1 clk = 1'b0;

  $display("-------------------- Computation completed
--------------------");

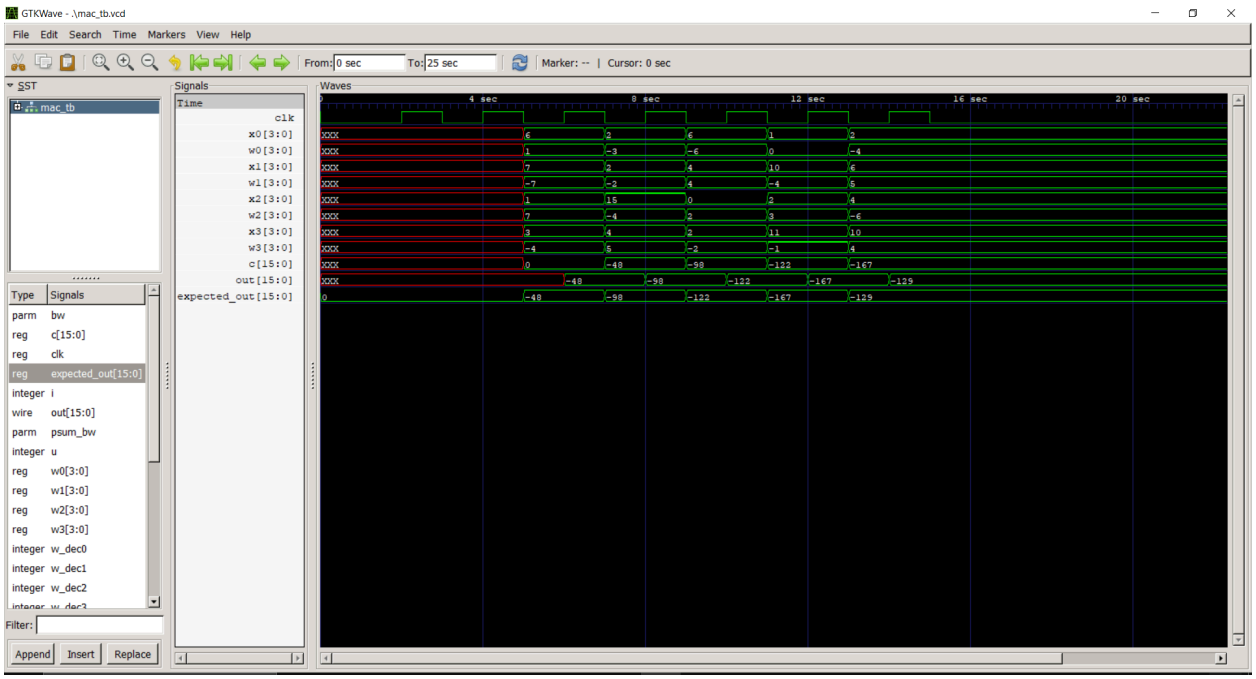
  #10 $finish;


end

endmodule
```

Q 3

Waveform:



mac.v:

```
// Created by prof. Mingu Kang @VVIP Lab in UCSD ECE department
// Please do not spread this code without permission
module mac (out, x0, w0, x1, w1, x2, w2, x3, w3, in);

parameter bw = 4;
parameter psum_bw = 16;

input unsigned [bw-1:0] x0;
input unsigned [bw-1:0] x1;
input unsigned [bw-1:0] x2;
input unsigned [bw-1:0] x3;
input signed [bw-1:0] w0;
input signed [bw-1:0] w1;
input signed [bw-1:0] w2;
input signed [bw-1:0] w3;

input signed [psum_bw-1:0] in;
```

```
output signed [psum_bw-1:0] out;

wire signed [bw:0] x0_signed;
wire signed [bw:0] x1_signed;
wire signed [bw:0] x2_signed;
wire signed [bw:0] x3_signed;
wire signed [2*bw:0] product0;
wire signed [2*bw:0] product1;
wire signed [2*bw:0] product2;
wire signed [2*bw:0] product3;

wire signed [2*bw+1:0] internal_sum0;
wire signed [2*bw+1:0] internal_sum1;

assign x0_signed = $signed({1'b0, x0});
assign x1_signed = $signed({1'b0, x1});
assign x2_signed = $signed({1'b0, x2});
assign x3_signed = $signed({1'b0, x3});

assign product0 = x0_signed * w0;
assign product1 = x1_signed * w1;
assign product2 = x2_signed * w2;
assign product3 = x3_signed * w3;

assign internal_sum0 = product0 + product1;
assign internal_sum1 = product2 + product3;

assign out = internal_sum0 + internal_sum1 + in;

endmodule
```

mac_wrapper.v:

```
// Created by prof. Mingu Kang @VVIP Lab in UCSD ECE department
// Please do not spread this code without permission
module mac_wrapper (
    clk,
    out,
    x0, x1, x2, x3,
    w0, w1, w2, w3,
```

```
    c
);

parameter bw = 4;
parameter psum_bw = 16;

output [psum_bw-1:0] out;
input  [bw-1:0] x0;
input  [bw-1:0] x1;
input  [bw-1:0] x2;
input  [bw-1:0] x3;
input  [bw-1:0] w0;
input  [bw-1:0] w1;
input  [bw-1:0] w2;
input  [bw-1:0] w3;
input  [psum_bw-1:0] c;
input  clk;

// Separate pipeline registers for each input channel
reg    [bw-1:0] x0_q, x1_q, x2_q, x3_q;
reg    [bw-1:0] w0_q, w1_q, w2_q, w3_q;
reg    [psum_bw-1:0] c_q;

// Structural instantiation matching the new mac design explicitly
mac #(.bw(bw), .psum_bw(psum_bw)) mac_instance (
        .x0(x0_q),
        .w0(w0_q),
        .x1(x1_q),
        .w1(w1_q),
        .x2(x2_q),
        .w2(w2_q),
        .x3(x3_q),
        .w3(w3_q),
        .in(c_q),
        .out(out)
);

always @ (posedge clk) begin
        x0_q <= x0;
        x1_q <= x1;
```

```
        x2_q <= x2;
        x3_q <= x3;

        w0_q <= w0;
        w1_q <= w1;
        w2_q <= w2;
        w3_q <= w3;

        c_q  <= c;
end

endmodule
```

mac_tb.v:

```
// Created by prof. Mingu Kang @VVIP Lab in UCSD ECE department
// Please do not spread this code without permission


module mac_tb;

parameter bw = 4;
parameter psum_bw = 16;

reg clk = 0;

reg  [bw-1:0] x0, x1, x2, x3;
reg  [bw-1:0] w0, w1, w2, w3;
reg  [psum_bw-1:0] c;
wire [psum_bw-1:0] out;
reg  [psum_bw-1:0] expected_out = 0;

integer w_file ; // file handler
integer w_scan_file ; // file handler

integer x_file ; // file handler
integer x_scan_file ; // file handler

integer x_dec0, x_dec1, x_dec2, x_dec3;
integer w_dec0, w_dec1, w_dec2, w_dec3;
```

```
integer i;
integer u;

function [3:0] w_bin ;
  input integer  weight ;
  begin

    if (weight>-1)
     w_bin[3] = 0;
    else begin
     w_bin[3] = 1;
     weight = weight + 8;
    end

    if (weight>3) begin
     w_bin[2] = 1;
     weight = weight - 4;
    end
    else
     w_bin[2] = 0;

    if (weight>1) begin
     w_bin[1] = 1;
     weight = weight - 2;
    end
    else
     w_bin[1] = 0;

    if (weight>0)
     w_bin[0] = 1;
    else
     w_bin[0] = 0;

  end
endfunction



function [3:0] x_bin ;
  input integer  weight ;
```

```
  begin
    for (u=bw-1; u>-1; u=u-1) begin
      if (weight>2**u-1) begin
        x_bin[u] = 1;
        weight = weight - 2**u;
      end
      else begin
        x_bin[u] = 0;
      end
    end
  end
endfunction

// Below function is for verification
function [psum_bw-1:0] mac_predicted;
    input [bw-1:0] x0;
    input [bw-1:0] x1;
    input [bw-1:0] x2;
    input [bw-1:0] x3;
    input signed [bw-1:0] w0;
    input signed [bw-1:0] w1;
    input signed [bw-1:0] w2;
    input signed [bw-1:0] w3;
    input signed [psum_bw-1:0] acc_in;

    reg signed [bw:0] x0_s, x1_s, x2_s, x3_s;
    reg signed [2*bw:0] p0, p1, p2, p3;

    begin
        x0_s = $signed({1'b0, x0});
        x1_s = $signed({1'b0, x1});
        x2_s = $signed({1'b0, x2});
        x3_s = $signed({1'b0, x3});

        p0 = x0_s * w0;
        p1 = x1_s * w1;
        p2 = x2_s * w2;
        p3 = x3_s * w3;

        mac_predicted = acc_in + p0 + p1 + p2 + p3;
```

```
    end
endfunction


mac_wrapper #(.bw(bw), .psum_bw(psum_bw)) mac_wrapper_instance (
        .clk(clk),
        .x0(x0), .x1(x1), .x2(x2), .x3(x3),
        .w0(w0), .w1(w1), .w2(w2), .w3(w3),
        .c(c),
        .out(out)
);


initial begin
  w_file = $fopen("b_data.txt", "r");  //weight data
  x_file = $fopen("a_data.txt", "r");  //activation

  $dumpfile("mac_tb.vcd");
  $dumpvars(0,mac_tb);

  #1 clk = 1'b0;
  #1 clk = 1'b1;
  #1 clk = 1'b0;

  $display("-------------------- Computation start --------------------");


  for (i=0; i<5; i=i+1) begin

    #1 clk = 1'b1;
    #1 clk = 1'b0;

    w_scan_file = $fscanf(w_file, "%d\n", w_dec0);
    w_scan_file = $fscanf(w_file, "%d\n", w_dec1);
    w_scan_file = $fscanf(w_file, "%d\n", w_dec2);
    w_scan_file = $fscanf(w_file, "%d\n", w_dec3);

    x_scan_file = $fscanf(x_file, "%d\n", x_dec0);
    x_scan_file = $fscanf(x_file, "%d\n", x_dec1);
    x_scan_file = $fscanf(x_file, "%d\n", x_dec2);
```

```
    x_scan_file = $fscanf(x_file, "%d\n", x_dec3);

    x0 = x_bin(x_dec0);
    x1 = x_bin(x_dec1);
    x2 = x_bin(x_dec2);
    x3 = x_bin(x_dec3);

    w0 = w_bin(w_dec0);
    w1 = w_bin(w_dec1);
    w2 = w_bin(w_dec2);
    w3 = w_bin(w_dec3);

    c = expected_out;

    expected_out = mac_predicted(x0, x1, x2, x3, w0, w1, w2, w3, c);

  end



  #1 clk = 1'b1;
  #1 clk = 1'b0;

  $display("--------------------- Computation completed
---------------------");

  #10 $finish;


end

endmodule
```

UCSD Linux Cloud: N/A. This project was done on a local setup.
Github: https://github.com/chaitany-raghav/ece284fa26/tree/my-solutions